Linear Neural Operator (LinearNO)

1 00

The Linear Neural Operator (LNO) is based on the low-rank decomposition of the operator kernel,

$$W(x,y) = \sum_{j=1}^{r} \phi_j(x)\psi_j(y). \tag{1}$$

This formulation is closely related to the unstacked DeepONet representation introduced by Lu et al. [1].

References

[1] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, pp. 218–229, 2021.

doi:10.1038/s42256-021-00302-5.

2 Introduction

A neural operator learns a mapping between function spaces. Let

$$\mathcal{G} : \mathcal{A} \to \mathcal{U} \tag{2}$$

be an operator that maps an input function

$$a(x) \tag{3}$$

to an output function

$$u(x) = \mathcal{G}(a)(x). \tag{4}$$

The objective is to construct a parameterized operator

$$\mathcal{G}_\theta \tag{5}$$

such that

$$\mathcal{G}_\theta(a) \approx \mathcal{G}(a). \tag{6}$$

Unlike a conventional neural network, the objective is therefore to learn a mapping between functions rather than only a mapping between fixed-dimensional vectors.

3 Integral Operator Formulation

Consider an integral operator

$$(\mathcal{K}v)(x) = \int_D W(x,y)v(y)\,dy, \tag{7}$$

where $W(x,y)$ is the operator kernel.
A general neural-operator layer can be written as

$$v_{l+1}(x) = \sigma\left(W_l v_l(x) + \mathcal{K}_l(v_l)(x) + b_l(x)\right). \tag{8}$$

Here W_l is a local linear transformation, while $\mathcal{K}_l$ provides the nonlocal interaction.

4 Low-Rank Kernel Representation

The defining assumption of LinearNO is that the kernel can be approximated by a finite-rank separable decomposition:

$$\boxed{W(x,y) = \sum_{j=1}^{r} \phi_j(x)\psi_j(y)}. \tag{9}$$

The integer r is the rank of the approximation.
The functions

$$\psi_j(y) \tag{10}$$

represent input-side basis functions, while

$$\phi_j(x) \tag{11}$$

represent output-side basis functions.
Substituting the decomposition into the integral operator gives

$$(\mathcal{K}v)(x) = \int_D \left[\sum_{j=1}^{r} \phi_j(x)\psi_j(y)\right] v(y)\,dy. \tag{12}$$

Interchanging the summation and integration,

$$(\mathcal{K}v)(x) = \sum_{j=1}^{r} \phi_j(x) \int_D \psi_j(y)v(y)\,dy. \tag{13}$$

Define

$$z_j = \int_D \psi_j(y)v(y)\,dy. \tag{14}$$

Then

$$\boxed{(\mathcal{K}v)(x) = \sum_{j=1}^{r} \phi_j(x)z_j}. \tag{15}$$

Thus the operator consists of two stages:

$$v(y) \longrightarrow \{z_j\}_{j=1}^{r} \longrightarrow u(x). \tag{16}$$

5 Neural Parameterization

The input-side basis functions can be generated by a neural network

$$\Psi_\theta(y) = \begin{bmatrix} \psi_1(y) \\ \psi_2(y) \\ \vdots \\ \psi_r(y) \end{bmatrix}. \tag{17}$$

Similarly, the output-side basis functions are generated by

$$\Phi_\theta(x) = \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \\ \vdots \\ \phi_r(x) \end{bmatrix}. \tag{18}$$

The operator can therefore be expressed as

$$(\mathcal{K}_\theta v)(x) = \Phi_\theta(x)^T \int_D \Psi_\theta(y) v(y)\, dy. \tag{19}$$

Define

$$\boldsymbol{z} = \int_D \Psi_\theta(y) v(y)\, dy. \tag{20}$$

Then

$$\boxed{(\mathcal{K}_\theta v)(x) = \Phi_\theta(x)^T \boldsymbol{z}}. \tag{21}$$

6 Discrete Formulation

Let the input domain be discretized at points

$$\{x_i\}_{i=1}^N \tag{22}$$

with quadrature weights

$$\{w_i\}_{i=1}^N. \tag{23}$$

The coefficient z_j becomes

$$z_j \approx \sum_{i=1}^N w_i \psi_j(x_i) v(x_i). \tag{24}$$

Therefore,

$$(\mathcal{K}v)(x_q) = \sum_{j=1}^r \phi_j(x_q) \sum_{i=1}^N w_i \psi_j(x_i) v(x_i). \tag{25}$$

Rearranging,

$$(\mathcal{K}v)(x_q) = \sum_{i=1}^N \left[\sum_{j=1}^r \phi_j(x_q) \psi_j(x_i) \right] w_i v(x_i). \tag{26}$$

Thus the discrete kernel is

$$\boxed{W(x_q, x_i) = \sum_{j=1}^{r} \phi_j(x_q)\psi_j(x_i)}. \tag{27}$$

7 Matrix Form

Define

$$\Phi_{qj} = \phi_j(x_q), \tag{28}$$

and

$$\Psi_{ij} = \psi_j(x_i). \tag{29}$$

Then

$$\Phi \in \mathbb{R}^{N_{\text{out}} \times r}, \tag{30}$$

and

$$\Psi \in \mathbb{R}^{N_{\text{in}} \times r}. \tag{31}$$

The kernel matrix is

$$K = \Phi\Psi^T. \tag{32}$$

With quadrature weights,

$$D_w = \text{diag}(w_1, \ldots, w_N), \tag{33}$$

and

$$\boxed{\boldsymbol{u} = \Phi\Psi^T D_w \boldsymbol{v}}. \tag{34}$$

The rank satisfies

$$\text{rank}(K) \leq r. \tag{35}$$

8 Computational Complexity

A dense kernel requires an $N_{\text{out}} \times N_{\text{in}}$ matrix.

Its direct application has complexity

$$\mathcal{O}\left(N_{\text{out}} N_{\text{in}}\right). \tag{36}$$

The low-rank representation instead computes

$$\boldsymbol{z} = \Psi^T D_w \boldsymbol{v}, \tag{37}$$

followed by

$$\boldsymbol{u} = \Phi \boldsymbol{z}. \tag{38}$$

The corresponding complexity is

$$\mathcal{O}\left(N_{\text{in}} r + N_{\text{out}} r\right). \tag{39}$$

For

$$r \ll N, \tag{40}$$

this provides a substantial reduction in computational cost.

9 LinearNO Layer

A LinearNO layer can be expressed as

$$v_{l+1}(x) = \sigma\left(W_l v_l(x) + \mathcal{K}_l(v_l)(x) + b_l(x)\right), \tag{41}$$

where

$$\mathcal{K}_l(v_l)(x) = \sum_{j=1}^{r} \phi_{l,j}(x) \int_D \psi_{l,j}(y) v_l(y)\, dy. \tag{42}$$

The rank-r kernel is

$$W_l(x, y) = \sum_{j=1}^{r} \phi_{l,j}(x) \psi_{l,j}(y). \tag{43}$$

10 Full Architecture

The input function is first lifted into a latent representation:

$$v_0(x) = P_\theta(a(x)). \tag{44}$$

It then passes through L LinearNO layers:

$$v_{l+1} = \sigma\left(W_l v_l + \mathcal{K}_l(v_l) + b_l\right). \tag{45}$$

The final latent representation is projected to the output:

$$\hat{u}(x) = Q_\theta(v_L(x)). \tag{46}$$

Hence,

$$\boxed{\hat{u} = Q_\theta \circ \mathcal{L}_L \circ \cdots \circ \mathcal{L}_1 \circ P_\theta(a)}. \tag{47}$$

11 Relation to DeepONet

DeepONet represents an operator using branch and trunk networks:

$$\hat{u}(x) = \sum_{j=1}^{r} b_j(a) t_j(x). \tag{48}$$

LinearNO has the analogous low-rank structure

$$\hat{u}(x) = \sum_{j=1}^{r} \phi_j(x) \int_D \psi_j(y) v(y)\, dy. \tag{49}$$

Thus the input function is first projected onto a finite set of learned functionals and the resulting coefficients are combined with learned output basis functions.

12 Training Objective

Given training pairs

$$\mathcal{D} = \left\{ \left(a^{(n)}, u^{(n)} \right) \right\}_{n=1}^{N_{\text{train}}}, \tag{50}$$

the prediction is

$$\hat{u}^{(n)} = \mathcal{G}_\theta \left(a^{(n)} \right). \tag{51}$$

The mean-squared-error loss is

$$\mathcal{L} = \frac{1}{N_{\text{train}}} \sum_{n=1}^{N_{\text{train}}} \left\| \hat{u}^{(n)} - u^{(n)} \right\|_2^2. \tag{52}$$

The relative L^2 error is

$$\mathcal{E}_{\text{rel}} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}. \tag{53}$$

13 PyTorch Implementation

```
import torch
import torch.nn as nn


class LowRankKernel(nn.Module):

    def __init__(
        self,
        coord_dim,
        channels,
        rank,
        hidden_dim=64
    ):
        super().__init__()

        self.rank = rank
        self.channels = channels

        self.psi = nn.Sequential(
            nn.Linear(
                coord_dim,
                hidden_dim
            ),
            nn.GELU(),
            nn.Linear(
                hidden_dim,
                hidden_dim
            ),
            nn.GELU(),
            nn.Linear(
                hidden_dim,
                channels * rank
            )
```

```
        )

        self.phi = nn.Sequential(
            nn.Linear(
                coord_dim,
                hidden_dim
            ),
            nn.GELU(),
            nn.Linear(
                hidden_dim,
                hidden_dim
            ),
            nn.GELU(),
            nn.Linear(
                hidden_dim,
                channels * rank
            )
        )

    def forward(
        self,
        input_coords,
        output_coords,
        x,
        weights=None
    ):

        B, N_in, C = x.shape
        N_out = output_coords.shape[1]

        psi = self.psi(
            input_coords
        )

        psi = psi.view(
            B,
            N_in,
            C,
            self.rank
        )

        phi = self.phi(
            output_coords
        )

        phi = phi.view(
            B,
            N_out,
            C,
            self.rank
        )

        if weights is not None:

            psi = (
                psi
                * weights.view(
                    1,
```

```
                    N_in,
                    1,
                    1
                )
            )

        z = torch.einsum(
            "bncr,bnc->bcr",
            psi,
            x
        )

        out = torch.einsum(
            "bncr,bcr->bnc",
            phi,
            z
        )

        return out


class LinearNOBlock(nn.Module):

    def __init__(
        self,
        channels,
        coord_dim,
        rank,
        hidden_dim=64
    ):
        super().__init__()

        self.kernel = LowRankKernel(
            coord_dim=coord_dim,
            channels=channels,
            rank=rank,
            hidden_dim=hidden_dim
        )

        self.local = nn.Linear(
            channels,
            channels
        )

        self.activation = nn.GELU()

    def forward(
        self,
        input_coords,
        output_coords,
        x,
        weights=None
    ):

        nonlocal_part = self.kernel(
            input_coords,
            output_coords,
            x,
```

```
            weights
        )

        local_part = self.local(x)

        if (
            input_coords.shape[1]
            ==
            output_coords.shape[1]
        ):
            out = (
                nonlocal_part
                +
                local_part
            )
        else:
            out = nonlocal_part

        return self.activation(out)


class LinearNO(nn.Module):

    def __init__(
        self,
        in_channels,
        out_channels,
        coord_dim=2,
        width=64,
        rank=32,
        layers=4
    ):
        super().__init__()

        self.input_projection = nn.Linear(
            in_channels + coord_dim,
            width
        )

        self.layers = nn.ModuleList(
            [
                LinearNOBlock(
                    channels=width,
                    coord_dim=coord_dim,
                    rank=rank,
                    hidden_dim=width
                )
                for _ in range(layers)
            ]
        )

        self.output_projection = nn.Sequential(
            nn.Linear(
                width,
                width
            ),
            nn.GELU(),
            nn.Linear(
```

```
                width,
                out_channels
            )
        )

    def forward(
        self,
        coords,
        x,
        weights=None
    ):

        x = torch.cat(
            [
                coords,
                x
            ],
            dim=-1
        )

        x = self.input_projection(x)

        for layer in self.layers:

            residual = x

            x = layer(
                coords,
                coords,
                x,
                weights
            )

            x = x + residual

        return self.output_projection(x)


if __name__ == "__main__":

    device = (
        "cuda"
        if torch.cuda.is_available()
        else "cpu"
    )

    batch_size = 4
    n_points = 256

    coords = torch.rand(
        batch_size,
        n_points,
        2,
        device=device
    )

    field = torch.randn(
        batch_size,
```

```
        n_points,
        1,
        device=device
    )

    model = LinearNO(
        in_channels=1,
        out_channels=1,
        coord_dim=2,
        width=64,
        rank=32,
        layers=4
    ).to(device)

    prediction = model(
        coords,
        field
    )

    print(
        "Input:",
        field.shape
    )

    print(
        "Prediction:",
        prediction.shape
    )
```

14 Training

A standard supervised training loop is:

```

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-3
)

criterion = nn.MSELoss()

for epoch in range(100):

    model.train()

    optimizer.zero_grad()

    prediction = model(
        train_coords,
        train_input
    )

    loss = criterion(
        prediction,
        train_target
    )

```

```
    loss.backward()

    optimizer.step()

    if epoch % 10 == 0:

        print(
            f"Epoch {epoch}: "
            f"Loss = {loss.item():.6e}"
        )
```

15 Summary

The defining equation of LinearNO is the low-rank kernel decomposition

$$\boxed{W(x,y) = \sum_{j=1}^{r} \phi_j(x)\psi_j(y)}. \tag{54}$$

The corresponding operator is

$$\boxed{(\mathcal{K}v)(x) = \sum_{j=1}^{r} \phi_j(x) \int_D \psi_j(y)v(y)\,dy}. \tag{55}$$

The discretized form is

$$\boxed{\boldsymbol{u} = \Phi\Psi^T D_w \boldsymbol{v}}. \tag{56}$$

The rank satisfies

$$\text{rank}(K) \leq r, \tag{57}$$

and the low-rank structure reduces the kernel application from quadratic scaling to approximately

$$\mathcal{O}(Nr) \tag{58}$$

when $r \ll N$.